Project Plan: ScoundrelJS

1. Project Overview

- **Project Name:** ScoundrelJS
- **Type:** JavaScript-based Web Game / Portfolio Project
- **Source Material:** Based on the solitaire card game "Scoundrel" by Zach Gage and Kurt Bieg.
- **Core Concept:** A digital implementation of the Scoundrel ruleset that emphasizes state management and extensibility. The project features a "tabletop" aesthetic and distinct mechanics for deck manipulation (the "Scoundrel Shuffle") and complex weapon logic.

2. Core Features & Scope

Initial Scope

- **Start Screen:** Simple interface with a "Start Game" button and a rules summary sidebar.
- **Game Loop:**
  - **Room Generation:** Draw cards from the Dungeon (deck) to fill a 4-slot room.
  - **Player Choice:** Either interact with 3 cards or "Run Away" (skip the room).
  - **Room Refill:** If played, refilling occurs when only 1 card remains.
- **Mechanics:**
  - **Health:** Starts at 20. Max 20.
  - **Weapons:** Binding usage with specific degradation rules.
  - **Potions:** Healing logic (Max 1 per turn).
  - **The Scoundrel Shuffle:** When running away, the player chooses the order in which the room cards are returned to the bottom of the deck.
- **End Game:**
  - **Win:** Deck is empty. Score = HP (+ last potion value).
  - **Loss:** HP reaches 0. Score = HP - (Sum of remaining monsters in deck and room).

Eventual Expansion

- **Main Menu:** Select different game variants or rule tweaks.
- **Visual Polish:** Advanced animations for shuffling and card movement.

2.1 Selected Tech Stack

Component	Choice	Rationale

Framework	**Angular (v18+)**	Using **Standalone Components** removes the need for `NgModules`, keeping the application lightweight. Angular's strict structure is ideal for separating game logic (Services) from the presentation layer (Components), ensuring the code remains clean as the ruleset grows complex.
Language	**TypeScript**	Native to Angular and mandatory for this project. TypeScript interfaces (e.g., `interface Card`, `interface GameState`) are critical for ensuring type safety during complex interactions, such as comparing Weapon values against Monster values or handling weapon degradation logic.
State Management	**Angular Signals**	**Signals** are the modern standard for state in Angular. They provide fine-grained reactivity without the boilerplate of RxJS or Redux. **Usage:** A `GameService` will hold `writable` signals for HP and Deck, and `computed` signals for derived state (e.g., "Is Game Over?" or "Current Score"). This ensures the UI updates the exact millisecond a value changes.
Templating	**Control Flow**	Angular's new control flow syntax (`@if`, `@for`) provides a clean, readable way to render the 4-card room and handle empty states (e.g., empty deck placeholders) with better performance than legacy `*ngIf/ngFor`.
Animation	**Angular Animations**	The built-in `@angular/animations` module is powerful for state-based transitions. We can define triggers (e.g., `trigger('cardState')`) to handle complex sequences like the "Shake" effect on invalid moves or the sliding motion of the "Scoundrel Shuffle" (moving cards from Room to Deck).

Styling	**Tailwind CSS**	Tailwind's utility classes allow for rapid prototyping of the tabletop grid layout. Its responsive utilities make it easy to stack the 4-card room vertically on mobile and horizontally on desktop without writing custom media queries.

3. Technical Architecture & Feasibility

3.1 Project Structure (Angular Standalone)

Follow a "feature-based" folder structure to keep logic and presentation coupled loosely but organized.

```
src/app/
├── core/
│   ├── models/         # TypeScript Interfaces (Card, GameState)
│   ├── services/       # GameService (Signal State)
│   └── utils/          # Deck generators, Shufflers
├── features/
│   ├── game-board/     # Smart Container (Layout, Orchestration)
│   ├── card/           # Dumb Component (Presentation)
│   └── hud/            # Stats (HP, Weapon info)
└── shared/
    ├── ui/             # Generic buttons, modals
    └── animations/     # Angular animation triggers
```

3.2 Data Models (`core/models`)

Agents should define these interfaces strictly to ensure type safety across the app.

- **`type Suit`**: `'hearts' | 'diamonds' | 'clubs' | 'spades'`
- **`type CardType`**: `'potion' | 'weapon' | 'monster'`
- **`interface Card`**:
  - `id: string` (Unique UUID for tracking animations)
  - `suit: Suit`
  - `rank: string` ('2'...'10', 'J', 'Q', 'K', 'A')
  - `value: number` (Calculated: 2-10, J=11, Q=12, K=13, A=14)
  - `isFaceUp: boolean`
- **`interface WeaponState`**:

- `damage: number` (The weapon's current attack power)
- `lastSlainValue: number` (The max value of the *previous* monster killed. Init at 99.)
- `sourceCard: Card` (For UI reference)

3.3 State Management (`GameService`)

The Service is the "Single Source of Truth." It must use **Angular Signals**.

State Signals (Writable):

- `deck`: `signal<Card[]>` (The draw pile).
- `room`: `signal<Card[]>` (The active 4 cards. `null` for empty slots).
- `discard`: `signal<Card[]>` (Graveyard).
- `playerHP`: `signal<number>` (Current health).
- `equippedWeapon`: `signal<WeaponState | null>`.
- `turnState`: `signal<'idle' | 'selecting' | 'resolving'>` (For UI locking).
- `canSkip`: `signal<boolean>` (Tracks if the previous room was skipped).

Computed Signals (Read-Only):

- `score`: `computed(() => playerHP() + ...)`
- `isGameOver`: `computed(() => hp <= 0 || (deckEmpty && roomEmpty))`
- `currentRoomDamage`: `computed()` (Sum of all monsters in the room—for "Run Away" calculations if needed).

Core Methods:

1. **`initializeGame()`**: Generates deck, filters red face cards, shuffles, resets HP.
2. **`drawRoom()`**: Moves cards from `deck` to `room` until 4 slots are full. Checks for "Not Enough Cards" edge case.
3. **`handleCardInteraction(cardId: string)`**:
    - **If Potion:** Check if `potionsUsed > 0`. If no, Heal. If yes, Discard.
    - **If Weapon:** Move current weapon to discard. Set new `equippedWeapon`.
    - **If Monster:** Calculate `damage = MonsterVal - WeaponDmg`. Check `MonsterVal <= LastSlain`. Apply damage. Update `LastSlain`.
4. **`scoundrelShuffle(order: Card[])`**: Accepts the remaining room cards in a specific

order. Pushes them to the **bottom** of the `deck` array. Toggle `canSkip` to `false`.

3.4 Component Architecture

1.

- **Responsibility:** Layout grid using Tailwind. Subscribes to `GameService` signals.
- **Logic:** Handles the "Run Away" logic (detecting swipe or click order) and passes the result to the Service.
- **Template:** Uses `@for` to loop through `room` signal.

2.

- **Inputs:** `[card]: Card`, `[isInteractable]: boolean`, `[feedbackState]: 'valid' | 'invalid' | 'selected'`.
- **Styling:**
  - **Container:** Aspect-ratio `2.5/3.5` (Poker card).
  - **Tailwind:** `border-red-500` (Hearts/Diamonds), `border-slate-800` (Clubs/Spades).
  - **Content:** Center Rank/Suit.
-
- **Animations:** specific `@trigger` for entering the room (slide-in) and leaving (slide-to-discard).

3.

- **Inputs:** `[hp]`, `[weapon]`, `[deckCount]`.
- **Responsibility:** Displays the health bar and the currently equipped weapon card.

3.5 Styling Strategy (Tailwind CSS)

Define a central color theme in `tailwind.config.js` to ensure consistency.

- **Colors:**
  - `scoundrel-bg`: `#fdfbf7` (Warm paper texture)
  - `scoundrel-ink`: `#2d2a2e` (Dark text)
  - `scoundrel-red`: `#e11d48` (Hearts/Diamonds)
  - `scoundrel-gold`: `#d97706` (Highlights/Weapons)

- **Grid Layout:**
    - Mobile: `flex flex-col` (Stack room vertically).
    - Desktop: `grid grid-cols-4 gap-4` (Room horizontal).

3.6 Implementation Phases (For Agent Direction)

1. **Phase A: The Engine (Service Layer)**
    - Create `GameService`.
    - Implement `Deck` generation and `Shuffle` (Fisher-Yates).
    - Write Unit Tests for `handleCardInteraction` (specifically the Weapon degradation logic).
2. **Phase B: The Table (UI Skeleton)**
    - Create `CardComponent` with static dummy data.
    - Create `GameBoard` grid with Tailwind.
    - Bind `GameService` signals to the DOM.
3. **Phase C: The Interaction (Game Loop)**
    - Connect click events to `handleCardInteraction`.
    - Implement the "Run Away" button and `scoundrelShuffle` logic.
    - Add "Game Over" and "Victory" modals.
4. **Phase D: Polish (Animation)**
    - Add Angular Animations for card draws and invalid shakes.

4. detailed Game Logic

A. The Setup

1. **Filter:** Remove Jokers, Red Face Cards (J, Q, K), and Red Aces from a standard 52-card deck.
2. **Shuffle:** Randomize the remaining cards.
3. **Init:** Set HP to 20.

B. The Turn Structure

1. **Check Condition:** If Deck is empty and Room has fewer than 4 cards (and can't be filled), Game Over (Win).
2. **Fill Room:** Draw until 4 cards are face-up.
3. **Player Action:**
    - **Option 1: Run Away (Skip)**
        - *Condition:* CanSkipRoom must be true.
        - *Input:* Player selects "Stack Left-to-Right" or "Stack Right-to-Left".

        - *Action:* All 4 room cards/ or remaining cards are pushed to the bottom of the Deck array in the chosen order.
        - *State Update:* CanSkipRoom = false.
    - **Option 2: Engage**
        - Player must click/select exactly 3 cards to resolve them or select less cards to resolve before running away.
        - *State Update:* CanSkipRoom = true.
2. **Resolve**:
    - Cards are resolved in the order they select the cards, the UI updating and animation playing as the player's round plays out.

C. Card Interaction Rules

- **Monsters (Clubs/Spades):**
    - *Values:* 2-10 = Face Value, J=11, Q=12, K=13, A=14.
    - **Unarmed Combat:** HP -= MonsterValue.
    - **Armed Combat:**
        - *Constraint:* Can only use weapon if MonsterValue <= Weapon.lastSlainValue. (Note: Initialize lastSlainValue to Infinity or a high number when weapon is first equipped).
        - *Damage:* Damage = MonsterValue - WeaponValue. If Damage < 0, take 0.
        - *Update:* Weapon.lastSlainValue = MonsterValue.
- **Weapons (Diamonds):**
    - Equip immediately.
    - Move currently equipped weapon (if any) to discard.
    - Set Weapon.damageValue = CardValue.
    - Reset Weapon.lastSlainValue to theoretical max (e.g., 99) to allow attacking any monster first.
- **Potions (Hearts):**
    - *Constraint:* Only 1 potion usable per turn.
    - *Action:* If valid, HP += CardValue. Cap HP at 20.
    - *Waste:* If a second potion is picked, it is discarded with no effect.

5. UI/UX Design (Tabletop Aesthetic)

Layout

- **Top Row:** The "Room" (4 card slots).
- **Middle Left:** The "Dungeon" (Face-down deck).
- **Middle Right:** The "Discard" (Face-down pile).
- **Bottom Center:** HUD area.
    - **HP Counter:** Large, clear display.
    - **Equipped Weapon:** Slot showing current weapon + "Last Slain" indicator.
    - **Slain Stack:** Visual pile of monsters killed by current weapon.

Feedback System

- **Visual Cues:**
  - Dim cards that have no effect or will just be discarded with (e.g., if Potion limit reached).
  - Highlight valid cards during combat selection.
- **Animations:**
  - "Shake" animation for invalid actions (e.g., trying to skip twice).
  - "Flash Red" on damage taken.
  - "Flash Green" on heal.

6. Development Roadmap

1. **Phase 1: Logic Core (MVP)**
   - Implement Deck, Card, and GameState classes in pure JS.
   - Simulate a full game in the browser console to verify weapon degradation and shuffle logic.
2. **Phase 2: The Interface**
   - Build HTML structure.
   - Connect DOM events to GameState methods.
   - Implement basic rendering (text-based cards).
3. **Phase 3: The "Scoundrel" Features**
   - Implement the "Run Away" UI (choosing stack order).
   - Add visual card assets (CSS or images).
4. **Phase 4: Polish**
   - Add animations.
   - Add Win/Loss screens with score calculation.
   - Responsive design adjustments.

Part 1: Potential Flaws & Shortcomings

1. The "End Game" Edge Case (The Dregs)

- **The Flaw:** The plan states: *"Refilling occurs when only 1 card remains."* and *"Draw until 4 cards are face-up."*
- **The Issue:** What happens when the deck has only 1 or 2 cards left? You cannot fill the room to 4.
- **The Fix:** You need a specific state handler for the final room. If the deck + current room < 4 cards, the "Run Away" option must be disabled (rules usually

imply you can't run from a partial room or the final boss). The game loop must handle a "Partial Room" state.

2. The "Barehanded" Fallback UX

- **The Flaw:** The plan says: *"Constraint: Can only use weapon if `MonsterValue <= Weapon.lastSlainValue`."*
- **The Issue:** The UI logic implies that if I click a strong monster while holding a weak weapon, the action is invalid (shake animation). However, the rules state: *"you must fight the Queen barehanded... the Weapon is not discarded."*
- **The Fix:** Clicking a monster that is "too strong" for the weapon shouldn't result in an error. It should result in a **modal or tooltip confirmation**: *"Weapon too weak! Fight barehanded for X damage?"*
- Alternate idea: Always have a modal to select between Weapon or barehanded use

3. Cognitive Load (The Math Problem)

- **The Flaw:** The plan relies on the user to calculate damage (`Monster - Weapon`).
- **The Issue:** While simple math, doing this repeatedly is tiring. In a digital port, the computer should do the heavy lifting.
- **The Fix:**
    - **Hover States:** When hovering over a monster, the card should visually change to show the *net damage* the player will take.
    - **Weapon UI:** The weapon should clearly display "Max Target: [Card Value]" so the user doesn't have to remember the "last slain" value.

4. The "Undo" Problem

- **The Flaw:** There is no mention of an Undo button.
- **The Issue:** Mis-clicks happen. Since no hidden information is revealed *during* the turn (you see all 4 cards at the start), there is no competitive reason to forbid undoing an action before the turn is committed.
- **The Fix:** Implement a `turnHistory` stack. Allow the user to "Reset Room" as long as they haven't drawn new cards yet. Have the user be able to select 3 cards and precalculate their turn